

NATIONAL ECONOMICS UNIVERSITY (NEU)
FACULTY OF DATA SCIENCE & FINANCIAL ENGINEERING (DSFE)

OFFICIAL SOLUTIONS & MARKING RUBRIC

Course: Fundamental Programming Concepts in Python (DSAI1003)

Unit: Part 01 Exam — Computers, Python Architecture, Errors, Algorithms & Basic I/O

Note for Instructors and TAs: This document provides full solutions, rationale for theoretical questions, trace tables, sample Python scripts, and an itemized marking rubric designed for fair and consistent grading.

Section A: Conceptual & Architectural Fundamentals (20 points)

2 points per question. Total = 20 points.

Q#	Key	Conceptual Explanation & Lecture Reference	Marks
Q1	B	The CPU fetches, decodes, and executes instructions. RAM holds active data; secondary storage stores persistent files.	2 pts
Q2	C	Primary Memory (RAM) is volatile; its content is immediately lost when electrical power ceases.	2 pts
Q3	B	A computer is general-purpose because it executes different software programs (ordered instructions and decisions).	2 pts
Q4	B	Python pipeline: Source code (.py) → Byte code (.pyc) → Virtual Machine (PVM) → Execution.	2 pts
Q5	B	CPUs cannot directly run high-level source text. It must be translated to bytecode and interpreted by PVM.	2 pts
Q6	B	Chapter 1 defines the three essential properties: Unambiguous, Executable, and Terminating.	2 pts
Q7	B	Pseudocode allows programmers to verify algorithm logic in human terms prior to syntactical implementation.	2 pts
Q8	C	The built-in <code>input()</code> function unconditionally returns a string (<code>str</code>), requiring type conversion.	2 pts
Q9	B	Canonical sequence: Understand problem → Algorithm/Pseudocode → Hand trace → Python code → Run/Debug.	2 pts
Q10	C	A logic error compiles and runs without crash, but yields incorrect outputs due to an erroneous algorithm/formula.	2 pts

Section B: Code Tracing, Output Prediction & Error Diagnosis (25 points)

Q11. Output Prediction: sep and end (5 points)**Solution & Marking Guide for Q11****Exact Output:**

```
2026-09-10 EXAM::PYTHON
DONE
```

Grading Breakdown:

- **2.5 pts:** 2026-09-10 correctly joined with dashes and ended with a single space (no newline).
- **1.5 pts:** EXAM::PYTHON printed on the same line right after the space, followed by a newline.
- **1.0 pt:** DONE printed on the second line.

Q12. String Operations vs. Numeric Arithmetic (5 points)**Solution & Marking Guide for Q12****Exact Answers:**

- Line 1 (print(a + b)): **52** (1.25 pts - string concatenation)
- Line 2 (print(c + d)): **7** (1.25 pts - integer addition)
- Line 3 (print(a * 3)): **555** (1.25 pts - string repetition 3 times)
- Line 4 (print(c * 3)): **15** (1.25 pts - integer multiplication)

Q13. Error Taxonomy and Classification (5 points)**Solution & Marking Guide for Q13**

- **A.** print("Total is:" total)
 - **Classification (1.0 pt):** Compile-Time Error / Syntax Error
 - **Reason (0.7 pt):** Missing comma separator between arguments inside the function call, violating grammar.
- **B.** val = int("3.14159")
 - **Classification (1.0 pt):** Exception (Run-time Error) [ValueError]
 - **Reason (0.7 pt):** The string contains a decimal point; int() cannot convert floating-point text directly.
- **C.** avg = score1 + score2 + score3 / 3
 - **Classification (1.0 pt):** Logic Error (Semantic Error)
 - **Reason (0.6 pt):** Operator precedence divides only score3 by 3. Program runs without crashing but produces wrong average.

Q14. Deconstructing Python Statements (5 points)**Solution & Marking Guide for Q14**

- The comment: `# Display computed final score with bonus` (1.0 pt)
- The built-in function name: `print` (1.0 pt)
- The string literal argument: `"Final Score:"` (1.0 pt)
- The arithmetic expression argument: `base_score + 10` (1.0 pt)
- The variable name: `base_score` (1.0 pt)

Q15. Tracing Formatted f-Strings (5 points)**Solution & Marking Guide for Q15****Exact Output:**

```
Item: USB Drive | Qty: 4  
Subtotal: $50.00  
Discount (10%): $5.00
```

Grading Breakdown:

- 1.5 pts:** Line 1 with exact text, pipe delimiter, and variable substitutions.
- 1.75 pts:** Line 2 showing `$50.00` formatted with 2 decimal places.
- 1.75 pts:** Line 3 showing `$5.00` (50.0×0.10) with 2 decimal places.

Section C: Algorithm Design & Pseudocode (25 points)

Task 1: Identify Inputs and Outputs (5 points)

Task 1 Solution

- **Input (2.5 pts):** minutes_ridden (ride duration, positive integer or float).
- **Outputs (2.5 pts):** Itemized breakdown: unlock fee, ride duration charge, subtotal, sales tax (8%), and total amount due.

Task 2: Write Structured Pseudocode (10 points)

Task 2 Model Pseudocode

```
Set UNLOCK_FEE to 1.50
Set MINUTE_RATE to 0.25
Set TAX_RATE to 0.08

Prompt user for ride duration in minutes
Read minutes_ridden
Convert minutes_ridden to numeric value

Set ride_cost to minutes_ridden * MINUTE_RATE
Set subtotal to UNLOCK_FEE + ride_cost
Set tax_amount to subtotal * TAX_RATE
Set total_fare to subtotal + tax_amount

Display "Unlock Fee: $", UNLOCK_FEE
Display "Ride Cost: $", ride_cost
Display "Subtotal: $", subtotal
Display "Tax (8%): $", tax_amount
Display "Total Amount Due: $", total_fare
```

Marking Rubric (10 pts):

- 2 pts: Clear constant and variable initialization.
- 2 pts: Input acquisition and type readiness.
- 4 pts: Mathematically correct sequential calculations (ride cost, subtotal, tax, final total).
- 2 pts: Complete and descriptive display statements.

Task 3: Algorithm Verification (5 points)

Task 3 Solution

- a. **Unambiguous (1.7 pts):** Every instruction uses explicit arithmetic operations (+, ×) without vague or subjective terms.
- b. **Executable (1.7 pts):** Each step consists of basic primitive operations (reading input, multiplication, addition, screen printing) supported by real hardware.
- c. **Terminating (1.6 pts):** The algorithm executes a deterministic, linear sequence of 9 steps with no infinite looping, halting immediately after displaying the receipt.

Task 4: Hand-Trace Verification for 20 Minutes (5 points)**Task 4 Solution**

- **Unlock Fee:** \$1.50 (1.0 pt)
- **Riding Charge:** $20 \times \$0.25 = \5.00 (1.0 pt)
- **Subtotal Before Tax:** $\$1.50 + \$5.00 = \$6.50$ (1.0 pt)
- **Tax Amount (8%):** $\$6.50 \times 0.08 = \0.52 (1.0 pt)
- **Final Total to Pay:** $\$6.50 + \$0.52 = \$7.02$ (1.0 pt)

Section D: Applied Programming in Python (30 points)

Problem 1: Commuter Fuel Cost Calculator (15 points)

Problem 1 Model Code

```
# Commuter Fuel Cost Calculator
# Step 1: Input with float conversion
distance = float(input("Enter distance (km): "))
consumption = float(input("Enter fuel consumption (L/100km): "))
price = float(input("Enter fuel price per liter: "))

# Step 2: Processing
fuel_consumed = (distance * consumption) / 100
total_cost = fuel_consumed * price

# Step 3: Output with f-strings
print("\nTrip Summary:")
print(f"Fuel consumed: {fuel_consumed:.2f} L")
print(f"Total cost: {total_cost:.2f}")
```

Marking Rubric (15 points):

- **Input & Type Casting (5 pts):** 3 inputs correctly requested with `float(input(...))`. Deduct 2 pts if student used `int`, deduct 4 pts if conversion omitted.
- **Processing / Formulas (5 pts):** Correct calculation of `fuel_consumed` (2.5 pts) and `total_cost` (2.5 pts).
- **Formatted Output (5 pts):** Correct output labels and f-string precision specifier `:.2f` (3 pts for format, 2 pts for clarity).

Problem 2: Event Duration Breakdown (15 points)

Problem 2 Model Code

```
# Event Duration Breakdown
# Step 1: Input with int conversion
total_seconds = int(input("Enter total seconds: "))

# Step 2: Processing with // and %
hours = total_seconds // 3600
remaining_seconds = total_seconds % 3600

minutes = remaining_seconds // 60
seconds = remaining_seconds % 60

# Step 3: Output
print("\nConverted Time:")
print(f"{hours} hours, {minutes} minutes, {seconds} seconds")
```

Marking Rubric (15 points):

- **Input & Casting (4 pts):** Prompting and casting input to `int`. Deduct 2 pts if cast to `float`.
- **Hours Extraction (4 pts):** Correctly using `// 3600` and extracting remainder with `% 3600`.
- **Minutes and Seconds Extraction (4 pts):** Correctly using `// 60` and `% 60`.
- **Output Formatting (3 pts):** Clear and structured print statement displaying hours, minutes, and seconds.

— END OF INSTRUCTOR RUBRIC —